种树(tree)

思路

对于测试点1~2 可以使用数组通过计数的方法，记录每颗树的位置，循环遍历数组，记录每个区间的空的的长度，寻找最大值

对于测试点3~4 可以数组储存每颗树的位置，循环遍历数组，计算每个区间的长度

对于其他测试点

1. **维护断点**: 我们需要一个数据结构来存储所有已经存在的分割点（包括公路的起点0和终点x）。为了能快速找到新种树位置 `j` 左右两边的已有断点，这个数据结构需要支持高效的查找和插入。`set` 是理想的选择，因为它们内部是有序的，并且插入和查找操作的平均时间复杂度都是 O(log k)
2. **维护区间长度**: 同时，我们需要另一个数据结构来存储所有当前存在的区间长度。这个数据结构的核心要求是能够快速地找到最小值。`multiset` 是一个非常合适的选择。它是一个允许重复元素的有序集合，可以让我们在 O(1) 时间内获取到最大/最小元素，并在 O(log k) 时间内插入和删除元素。

参考程序-40分

```cpp
#include <bits/stdc++.h>

using namespace std;

vector<int> vt;
int n,k;

int main()
{
    cin>>n>>k;
    vt.push_back(0);
    vt.push_back(n);
    for(int i=1;i<=k;i++)
    {
        int x;
        cin>>x;
        vt.push_back(x);
        sort(vt.begin(),vt.end());
        int minn=n;
        for(int j=0;j<vt.size()-1;j++)
        {
            minn=min(minn,vt[j+1]-vt[j]);
        }
        cout<<minn<<" ";
    }
    return 0;
}
```

余博士教编程

参考程序-100分

```cpp
#include <bits/stdc++.h>

using namespace std;

const int N = 2e5 + 10;
long long a[N];

int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    multiset<long long> ms;
    long long x;
    int n;
    cin >> x >> n;
    set<long long> s;
    s.insert(0);
    s.insert(x);
    ms.insert(x);

    for(int i = 0;i < n;i ++)
    {
        cin>>a[i];
        long long j = a[i];
        auto it = s.lower_bound(j);
        long long r = *it, l = *(--it);
        s.insert(a[i]);
        ms.erase(ms.find(r - l));
        ms.insert(j - l);
        ms.insert(r - j);
        cout << *ms.begin() << " ";
    }
    return 0;
}
```

漂流(drifting)

思路

对于测试点1-2，$min_i = 1, max_i = n$,这一条件说明所有人可以相互组合，所以我们可以通过搜索的方法，枚举所有船乘坐人数的可能，求出最小的费用。

对于其他测试点，通过搜索的方法，枚举所有人数组合的可能，对于每一种情况要进行验证，可以通过优先队列的优化，对于每一个划分的情况，实现要min_i最小值满足情况，将满足情况的min_i放入优先队列，优先队列根据max_i从小到大排列，每次取出枚举小船人数个队首元素，计算所有符合条件情况下的最小费用。

余博士教编程

参考程序-20分

```cpp
#include<bits/stdc++.h>
using namespace std;

int n,ans=INT_MAX;
int w[105],l[105],r[105];
vector<int> vt;

int calc()
{
    int res=0;
    for(int i=0;i<vt.size();i++)
    {
        res+=w[vt[i]];
    }
    return res;
}

void dfs(int sum,int limit)
{
    if(sum==n)
    {
        ans=min(ans,calc());
        return;
    }
    for(int i=limit;i<=n-sum;i++)
    {
        vt.push_back(i);
        dfs(sum+i,i);
        vt.pop_back();
    }
}

int main() {
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>w[i];
    }
    for(int i=1;i<=n;i++)
    {
        cin>>l[i]>>r[i];
    }
    dfs(0,1);
    if(ans==INT_MAX) cout<<-1;
    else cout<<ans;
    return 0;
}
```

余博士教编程

参考程序-100分

```cpp
#include<bits/stdc++.h>
using namespace std;

int n,ans=INT_MAX;
int w[105],l[105],r[105];
vector<int> vt,lim[105];

int calc()
{
    int res=0;
    int k=1;
    priority_queue <int,vector <int>,greater <int>> peo;
    for(int i=0;i<vt.size();i++)
    {
        int x=vt[i];
        while(k<=x)
        {
            for(int j=0;j<lim[k].size();j++)
            {
                peo.push(lim[k][j]);
            }
            k++;
        }
        if((int)peo.size() < x || (!peo.empty() && peo.top() < x))
            return INT_MAX;
        for(int j=1;j<=x;j++) peo.pop();
        res+=w[x];
    }
    return res;
}

void dfs(int sum,int limit)
{
    if(sum==n)
    {
        ans=min(ans,calc());
        return;
    }
    for(int i=limit;i<=n-sum;i++)
    {
        vt.push_back(i);
        dfs(sum+i,i);
        vt.pop_back();
    }
}

int main() {
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>w[i];
    }
    for(int i=1;i<=n;i++)
```

```
    {
        cin>>l[i]>>r[i];
        lim[l[i]].push_back(r[i]);
    }
    dfs(0,1);
    if(ans==INT_MAX) cout<<-1;
    else cout<<ans;
    return 0;
}
```

字符侦探(character)

思路

对于测试点1~2 字符串是同一个字符，那么以第i个字符结尾的回文串的长度就为i

对于测试点3~4 字符串的长度在100范围内，可以通过枚举每一个以第i个字符结尾的字符串，验证是否为回文串，若为回文串记录最大值

对于测试点5~6 使用马拉车算法

为了统一处理奇数和偶数长度的回文串，我们将原字符串 `s` (如 `aabac`) 变换为 `t` (`^#a#a#b#a#c#$`)。`^` 和 `$` 是哨兵字符，防止越界。

`p[i]` 存储在变换后的字符串 `t` 中，以 `t[i]` 为中心的最长回文串的半径。

算法通过维护一个当前已发现的回文串能到达的最右边界 `mr` 和其中心 `ml`，来加速后续 `p[i]` 值的计算，从而实现 O(n) 的时间复杂度。

对于字符，循环枚举前i个字符为中心的回文串找到最长回文串输出长度。

对于其他测试点 基于5-6测试点的马拉车算法，先将**所有**回文串 `[start, end]` 存入一个数组 `q`，然后循环变量每个字符，对于每个字符回文串的判断，通过遍历所有回文串，在i位置之前的都移除，并输出最优回文串长度。

参考程序-20分

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    string s;
    cin>>s;
    int l=s.size();
    for(int i=1;i<=l;i++)
    {
        cout<<i<<" ";
    }
    return 0;
}
```

参考程序-40分

```cpp
#include <bits/stdc++.h>
using namespace std;

string s;
bool check(int l,int r)
{
    while(l<r)
    {
        if(s[l]==s[r])
        {
            l++;
            r--;
        }
        else
        {
            return false;
        }
    }
    return true;
}
int main()
{
    cin>>s;
    int l=s.size();
    for(int i=0;i<l;i++)
    {
        int maxn=1;
        for(int j=0;j<i;j++)
        {
            if(check(j,i))
            {
                maxn=max(maxn,i-j+1);
            }
        }
        cout<<maxn<<" ";
    }
    return 0;
}
```

参考程序-60分

```cpp
#include<bits/stdc++.h>
using namespace std;
const int maxn = 1e7 + 5;
char s[maxn], t[maxn << 1];
int n, m, p[maxn << 1];
int manacher()
{
    n = strlen(s + 1);
    t[0] = '^'; //扩展原字符串
    t[1] = '#';
    for(int i = 1; i <= n; i++)
```

余博士教编程

```
    {
        t[i << 1] += s[i];
        t[i << 1 | 1] = '#';
    }
    t[(n << 1) + 2] = '$';
    m = n << 1;
    int ans = 0;
    for(int i = 2, ml = 0, mr = 0; i <= m; i++) //只剩下奇数长度的情况了
    {
        p[i] = 1;
        if(i <= mr)
        {
            p[i] = min(mr - i, p[ml + mr - i]);
        }
        while(t[i - p[i]] == t[i + p[i]]) p[i]++;
        if(i + --p[i] > mr)
        {
            ml = i - p[i], mr = i + p[i];
        }
        ans = max(ans, p[i]);
    }
    return ans;
}
bool check(int l, int r) //判断子串[l,r]是否是回文
{
    return p[l + r] >= r - l + 1;
}
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> s + 1;
    manacher();
    for(int i=1;i<=n;i++)
    {
        int ans=1;
        for(int j=1;j<i;j++)
        {
            if(check(j,i))
            {
                ans=i-j+1;
                break;
            }
        }
        cout<<ans<<" ";
    }
    return 0;
}
```

参考程序-100分

```
#include<bits/stdc++.h>
using namespace std;
const int maxn = 1e7 + 5;
char s[maxn], t[maxn << 1];
int n, m, p[maxn << 1];
```

```
int manacher()
{
    n = strlen(s + 1);
    t[0] = '^'; //扩展原字符串
    t[1] = '#';
    for(int i = 1; i <= n; i++)
    {
        t[i << 1] += s[i];
        t[i << 1 | 1] = '#';
    }
    t[(n << 1) + 2] = '$';
    m = n << 1;
    int ans = 0;
    for(int i = 2, ml = 0, mr = 0; i <= m; i++) //只剩下奇数长度的情况了
    {
        p[i] = 1;
        if(i <= mr)
        {
            p[i] = min(mr - i, p[ml + mr - i]);
        }
        while(t[i - p[i]] == t[i + p[i]]) p[i]++;
        if(i + --p[i] > mr)
        {
            ml = i - p[i], mr = i + p[i];
        }
        ans = max(ans, p[i]);
    }
    return ans;
}
bool check(int l, int r) //判断子串[l,r]是否是回文
{
    return p[l + r] >= r - l + 1;
}
int l=1,r;
pair<int, int>q[maxn];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> s + 1;
    manacher();
    for(int i = 2; i <= m; i++)
    {
        if(p[i])
        {
            q[++r] = {(i - p[i] + 1) / 2, (i + p[i] - 1) / 2};
        }
        if(i % 2 == 0)
        {
            int j = i / 2;
            while(l <= r && q[l].second < j) l++;
            cout << j - (q[l].first + q[l].second - j) + 1 << " ";
        }
    }
    return 0;
}
```

余博士教编程

硬币(coin)

思路

对于测试点1 直接计算与根节点相连的路径的平均值即可，$K=2$ 相当于只有一个硬币可以移动一次，就是往相邻的节点移动，可能的路径也就是两条，相加之后除以2就行，注意相加之后的和可能为奇数，所以需要乘2的在模998244353意义下的逆元。

对于测试点2~3 此时树的形态是链状，此时有两种可能性，

一种是$N \geq K$，此时不需要考虑双倍得分的情况，此时硬币会在深度较浅的K个节点上，每个硬币产生的得分，就是该硬币到根节点上路径上的得分和，因此需要在dfs的过程中将这个得分和记录下来，记一个数组 sum, $sum[i]$表示节点i到根节点路径上的得分和，初始化$sum[1]=0$

一种是$N \leq K$，需要考虑双倍得分情况，此时可以特殊计算在叶子节点时的答案，我们知道会有 $K-N-1$个节点可以到达叶子节点，我们记这个数量为 k ，则叶子节点上会有 $k-1$个硬币被移除，这些硬币会产生双倍得分，记叶子节点为v，所以就会得到 $2\times(k-1)\times sum[v]+sum[v]$ 的得分，化简后就是 $（2\times k-1）\times sum[v]$的得分。

对于其他测试点 考虑动态规划，记 $dp[u][k]$ 为在节点 u 有 k 个硬币获得的期望，虽然实际上每个节点上不会存在多于一个硬币，但我们可以考虑把这些硬币先堆在这里，一次性一起移动。

当 节点 u 为 叶子节点时，可以计算当前位置下硬币产生的得分。在叶子节点上且当前硬币数大于 1 时，得分就是 $2\times k-1\times sum[u]$

当 节点 u 为 有一个子节点时，记子节点为$s1$，$dp[u][k]=sum[u]+dp[s1][k-1]$， 因为只有一个字节的，$k-1$个硬币一定都会移动到$s1$上。

当 节点 u 为 有两个子节点时，记子节点为$s1,s2$，此时需要考虑$k-1$个硬币的分配情况，设进入子节点$s1$的硬币有i个，则该情况的概率$np=\frac{C(k-1,i)}{2^{k-1}}$，其中$C(k-1,i)$表示组合数，$dp[u][k]=sum[u]+\sum_{i=0}^{k-1}np\times(dp[s1][i]+dp[s2][k-1-i])$，因此对于所有的 $i\in[1,k-1]$，都需要计算出$dp[u][i]$，所以需要执行 k 次 dfs 的过程。

参考程序-30分

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 500+10;
int n, k;
const int mod=998244353;
long long dp[N][N];
long long sum[N];
vector<pair<int,int>> e[N];
long long qpow(long long a,long long b)
{
    long long ans=1;
    while(b)
    {
        if(b%2)ans=ans*a%mod;
        a=a*a%mod;
        b/=2;
    }
    return ans;
}
long long ans=0;
```

```cpp
void dfs(int u,int k,int fa)
{
    if(k==0)return;
    int cnt=0;
    for(auto now:e[u])
    {
        int v=now.first;
        int w=now.second;
        if(v==fa)continue;
        sum[v]=(sum[u]+w)%mod;
        cnt++;
        dfs(v,k-1,u);
    }
    if(cnt==0)ans+=(2*k%mod*sum[u]%mod-sum[u]+mod)%mod;
    else ans=(ans+sum[u])%mod;
}
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> k;
    for(int i=1;i<n;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        e[u].push_back({v,w});
        e[v].push_back({u,w});
        if(k==2&&(u==1||v==1))ans+=w;
    }
    if(k==2)ans=ans*qpow(2,mod-2)%mod;
    else
    {
        dfs(1,k,0);
    }
    cout<<ans<<"\n";
    return 0;
}
```

参考程序-100分

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 500+10;
int n, k;
const int mod=998244353;
long long dp[N][N];
long long sum[N];
vector<pair<int,int>> e[N];
long long jc[N];
long long inv[N];
long long two[N];
long long qpow(long long a,long long b)
{
    long long ans=1;
    while(b)
    {
```

余博士教编程

```
        if(b%2)ans=ans*a%mod;
        a=a*a%mod;
        b/=2;
    }
    return ans;
}
long long C(long long n,long long m)
{
    if(n<m)return 0;
    return jc[n]*inv[m]%mod*inv[n-m]%mod;
}
void dfs(int u,int k,int fa)
{
    if(k==0)return;
    if(dp[u][k])return;
    vector<int> son;
    for(auto now:e[u])
    {
        int v=now.first;
        int w=now.second;
        if(v==fa)continue;
        sum[v]=(sum[u]+w)%mod;
        son.push_back(v);
        dfs(v,k-1,u);
    }
    dp[u][k]=sum[u]%mod;
    if(son.size()==2)
    {
        int s1=son[0];
        int s2=son[1];
        for(int i=0;i<k;i++)
        {
            long long np=C(k-1,i)*two[k-1]%mod;
            dp[u][k]+=np*(dp[s1][i]+dp[s2][k-i-1]%mod)%mod;
            dp[u][k]%=mod;
        }
    }else if(son.size()==1)
    {
        int s1=son[0];
        dp[u][k]+=dp[s1][k-1];
        dp[u][k]%=mod;
    }else if(k>1)dp[u][k]=(2*k%mod*sum[u]%mod-sum[u]+mod)%mod;
}
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> k;
    jc[0]=1;
    two[0]=1;
    two[1]=qpow(2,mod-2);
    for(int i=1;i<=k;i++)
    {
        jc[i]=jc[i-1]*i%mod;
        two[i]=two[i-1]*two[1]%mod;
    }
```

余博士教编程

```
    inv[k]=qpow(jc[k],mod-2);
    for(int i=k;i>0;i--)
    {
        inv[i-1]=inv[i]*i%mod;
    }
    for(int i=1;i<n;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        e[u].push_back({v,w});
        e[v].push_back({u,w});
    }
    for(int i=1;i<=k;i++)dfs(1,i,-1);
    cout<<dp[1][k]<<"\n";
    return 0;
}
```

余博士教编程